SimpleVLA-RL

视觉语言动作模型的在线强化学习

模型改造、轨迹采样与 GRPO 优化

作者：Yang kunpeng

目录

第一章 VLA 模型改造与在线轨迹采样 **1**

1.1 面向强化学习的 OpenVLA-OFT 改造 1

1.2 Interactive VLA Rollout 3

第二章 GRPO 强化学习与探索增强 **5**

2.1 Outcome Reward 与 Group Advantage 5

2.2 Token-level GRPO 策略优化 6

2.3 探索增强与完整训练闭环 8

第 1 章

VLA 模型改造与在线轨迹采样

SimpleVLA-RL 以 SFT 初始化的 VLA 为起点，在机器人环境中采集交互轨迹，并依据任务结果进行 GRPO 更新。策略必须能够随机采样动作并计算相应概率；环境必须执行动作并返回任务完成信息。二者分别提供策略梯度所需的动作概率和奖励信号。

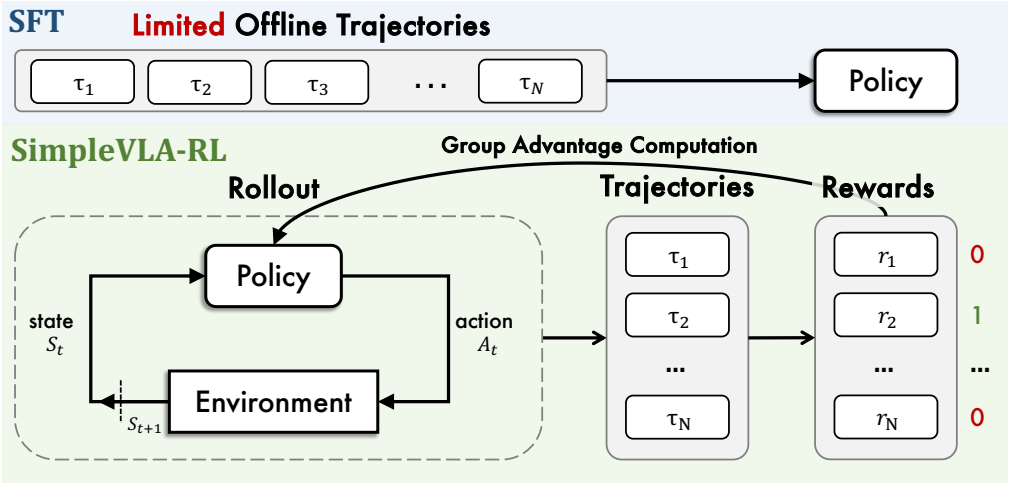


图 1.1: 模型生成操作轨迹，仿真环境执行动作并返回任务奖励，GRPO 利用采样动作概率更新策略。

1.1 面向强化学习的 OpenVLA-OFT 改造

原始策略与改造后策略。OpenVLA-OFT 以图像和语言任务为条件，利用视觉编码器及 LLaMA2-7B 骨干产生表示，再通过动作头一次预测多个连续动作。官方版本使用 MLP 回归连续动作，并采用 L_1 目标学习专家动作。SimpleVLA-RL 保留其并行动作解码与动作块设计，使用 LLaMA2 输出头预测离散动作 *Token*，以交叉熵进行监督微调。输入也有所调整：不使用腕部相机图像；LIBERO 中使用单视角图像和任务文本，RoboTwin 设置还使用机器人本体状态。输入配置及动作头的变化使官

方连续回归版本的下游检查点不能直接用作该方法的初始策略，因此需要重新进行SFT。

设一次观测记为

$$s_t = (o_t^{\text{vis}}, o_t^{\text{prop}}, \ell), \tag{1.1}$$

其中 o_t^{vis} 是相机观测，o_t^{prop} 是可能存在的本体状态，ℓ 是任务指令；特定实验没有本体输入时省去该项。论文定义的原子机器人动作是 $a_t \in \mathbb{R}^d$，典型的末端执行器控制使用 $d = 7$：三维位移、三维姿态增量与夹爪控制。模型每次输出长度为 H 的动作块

$$C_t = (a_t, a_{t+1}, \ldots, a_{t+H-1}) \in \mathbb{R}^{H\times d}. \tag{1.2}$$

t 表示环境控制时间，H 表示动作块的控制步数，而非任务轨迹的总长度。LIBERO设置使用 $H = 8$，RoboTwin 使用 $H = 25$。对于 $H = 8, d = 7$，一个 Chunk 有 $8 \times 7 = 56$ 个标量动作分量。

从连续分量到离散 Token。SimpleVLA-RL 对每个动作分量使用具有 256 个候选值的离散 Token 概率表示。用 Q 表示动作分量的量化映射，D 表示反量化，可写成

$$z_{t,h,j} = Q(a_{t+h,j}), \qquad \hat{a}_{t+h,j} = D(z_{t,h,j}), \quad z_{t,h,j} \in \{0, \ldots, 255\}. \tag{1.3}$$

$h = 0, \ldots, H-1$ 指动作块内部的控制步，$j = 1, \ldots, d$ 指动作维度。Q, D 表示动作量化和反量化操作，并非可训练网络。具体量化区间、夹爪符号和反归一化约定由所用数据处理和控制器决定，不能从上述通式直接推断真实机器人中的物理幅度。

一个动作块对应 Token 矩阵

$$Z_t = (z_{t,h,j}) \in \{0, \ldots, 255\}^{H\times d}. \tag{1.4}$$

动作头为每个待预测位置产生 256 维 logits。以 q 记动作块展平后的一个 Token 位置，令 $\boldsymbol{u}_{t,q} = f_\theta(s_t)_q \in \mathbb{R}^{256}$；训练时的温度采样分布可写为

$$\pi_\theta(z_{t,q} = k \mid s_t, q; T) = \frac{\exp(u_{t,q,k}/T)}{\sum_{k'=0}^{255} \exp(u_{t,q,k'}/T)}. \tag{1.5}$$

T 是采样温度。升高 T 通常会使分布平缓，增加原本低概率动作 Token 被选中的机会。与直接输出一个连续均值不同，离散策略既能随机采样，又能对实际采样到的同一个 Token 计算 $\log \pi_\theta$，这是后续重要性比值的基础。

并行动作解码表示模型在一次前向中形成一个动作块的多个位置输出，不意味着执行时每个原子动作都重新调用 VLA，也不能直接等同于普通 LLM 在单个 Chunk 内逐 Token 自回归生成。若用位置 $q = (h, j)$ 索引展开后的 Token，数学上的逐位置损失只需读取对应的条件概率。具体概率分解方式以模型输出及采样器的接口为准。

SFT 初始化。设 D_{demo} 为专家演示，监督目标为量化后的专家动作 z_q^*。模型以交叉熵最大化示范动作在对应位置的概率：

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(s,Z^*)\sim D_{\text{demo}}} \left[\frac{1}{|Z^*|} \sum_{q=1}^{|Z^*|} \log \pi_\theta(z_q^* \mid s, q; T = 1) \right]. \tag{1.6}$$

SFT 为在线采样提供初始完成任务的能力；如果训练一开始根本生成不了成功轨迹，只有 0/1 结果奖励的 GRPO 就难以获得可比较的正样本。

1.2 Interactive VLA Rollout

闭环动作生成。给定初始环境观测 s_0，策略采样第一个动作块 Z_0，解码为 C_0。模拟器逐个执行块内动作，期间物体、机器人位姿与相机观测随物理交互变化。完成该块后，模型读取新的观测 s_H 并生成下一块：

$$s_0 \xrightarrow{\pi_\theta} C_0 \xrightarrow{\text{Env}} s_H \xrightarrow{\pi_\theta} C_H \xrightarrow{\text{Env}} s_{2H} \longrightarrow \cdots. \tag{1.7}$$

不同于 LLM 只在内部扩展一个文本序列，VLA 的后续观测必须来自真正的环境状态转移。完整轨迹 τ_i 包含一连串观测、采样动作 Token、实际控制动作、终止状态及最后的成功标记。

一组轨迹而非一个动作块。GRPO 中的 Group 是相同任务条件下采集的多条完整轨迹，论文主要设置使用 $G = 8$。每条完整轨迹可包含不同数量的动作块，记第 i 条轨迹的有效动作 Token 总数为 L_i。例如 LIBERO 中一条轨迹若完整执行三个 8×7 的 Chunk，则有 $L_i = 168$ 个有效 Action Token；另一个只执行两个半 Chunk 的任务可能具有不同 L_i。Group Size G、Chunk Length H 和轨迹 Token Length L_i 是不同概念，不能混为一谈。

并行 Rollout 使用环境进程池组织交互：活跃环境提供观测，模型采样动作，环境执行动作并返回新观测与终止信号，完成的环境退出活跃集合。核心控制逻辑可概括为：

```
active = initialize_environments(task_batch, repeats=G)
while active:
    obs = read_observations(active)
    chunks, sampled_tokens = policy.sample(obs, temperature=T)
    next_obs, done, success = step_envs(active, chunks)
    save_rollout(obs, sampled_tokens, done, success)
    active = remove_finished(active, done)
```

`step_envs` 内部逐原子动作执行。环境接口还需要处理不同环境的任务长度、异步返回、恢复动作尺度与夹爪约定。官方 `rob_rollout.py` 中 LIBERO 环境工作线程明确逐个执行 Chunk 内动作，并跟踪 `finish_step`、`active` 与 `complete`。这些变量分别参与有效时间步统计、活跃环境筛选及任务成功判定。

训练数据与有效 Token。对旧策略 $\pi_{\theta_{\text{old}}}$ 采集的动作位置，训练需要保存观测（或重算网络输入所需的张量）、实际采样 Token、旧策略 log probability、终止信息以及奖励。对存在 Padding 的轨迹，定义

$$m_{i,q} = \begin{cases} 1, & q \text{ 对应已执行的有效动作 Token,} \\ 0, & q \text{ 是任务终止后的补齐位置.} \end{cases} \tag{1.8}$$

后续的 Advantage 和 Policy Loss 只对 $m_{i,q} = 1$ 的位置计入。官方 `dp_rob.py` 依据 `finish_step` 与每步 Token 数量生成有效 Mask，并在策略更新时使用该 Mask 屏蔽尾部位置；这避免模型因人为补齐动作而收到错误梯度。

Rollout 采样与策略评估。训练期使用随机采样，以 $T = 1.6$ 扩大轨迹差异；评估期使用 Greedy Sampling。温度改变的是执行策略的采样分布，反向计算旧/新策略对同一 Token 的概率时，需要与 Rollout 概率的定义一致。官方 Actor 的 `compute_log_prob` 显式读取来自批数据的 `temperature` 元信息，用于避免采样与概率重算不一致。评估阶段不需要采样 Group、计算 Advantage 或执行参数更新；机器人仍按“观察、预测 Chunk、执行、再观察”的闭环过程工作。

第 2 章

GRPO 强化学习与探索增强

SimpleVLA-RL 不训练独立的价值网络。它使用同一任务下多条轨迹的最终完成情况计算相对优势，并将轨迹级优势作用于其中每一个有效动作 Token。训练因而分成两个彼此依赖的计算阶段：环境决定轨迹的好坏，策略网络决定如何改变实际采样动作的概率。探索增强在产生轨迹、筛选数据和约束更新三个阶段分别起作用。

2.1 Outcome Reward 与 Group Advantage

结果奖励。第 i 条轨迹的最终奖励是一个标量

$$R_i = \begin{cases} 1, & \text{is_successful}(\tau_i) = \text{true}, \\ 0, & \text{其他情况}. \end{cases} \tag{2.1}$$

模型并不在每个抓取子阶段或每个控制步上额外学习一个手工设计的密集奖励。成功轨迹的全部有效 Token 将在后续获得同一个轨迹优势；这是一种信用分配方式，并不表示环境在每个时刻都返回奖励 1。任务完成条件由具体仿真环境的判定器决定，奖励不需要对环境动力学求导。

组内相对优势。一组 G 条轨迹的奖励为 $(R_1, \ldots, R_G)$，定义

$$\bar{R} = \frac{1}{G}\sum_{i=1}^{G} R_i, \qquad \hat{A}_i = \frac{R_i - \bar{R}}{\text{std}(R_1, \ldots, R_G) + \varepsilon_A}. \tag{2.2}$$

ε_A 是避免标准差为零的稳定项；论文的数学表达式未显式写出稳定项；官方 `compute_grpo_outcome_advantage` 在分母加入数值稳定项。GRPO 的组内比较不需要训练额外 Critic。奖励 R_i 是标量，$\hat{A}_i$ 仍是标量；将其扩展到 Token 维度时，$A_{i,q} = m_{i,q}\hat{A}_i$。

例如一组八条轨迹奖励为

$$(R_1, \ldots, R_8) = (1, 0, 1, 0, 1, 0, 1, 0), \qquad \bar{R} = 0.5. \tag{2.3}$$

用总体标准差 $\sigma = 0.5$ 计算，则成功和失败的优势分别是 $+1$、-1。PyTorch 默认 `torch.std` 使用样本标准差，该组数据的样本标准差是 $\sqrt{2/7} \approx 0.5345$，于是对应值约为 ± 0.9354。两种标准差约定不影响正负方向，但会影响实际梯度尺度；把公式译成代码时应核对实现采用的约定。

组内没有差异时会怎样。若八条轨迹全部失败，$R_i = \bar{R} = 0$，组内优势都为零；全部成功时也一样。在当前 Outcome GRPO 设定下，这样的组无法通过相对奖励区分好坏。这一性质决定了 Dynamic Sampling 的设计需求，也解释了为什么有一定成功率的 SFT 初始策略尤为重要。

官方 `core_algos.py` 中的相关计算首先对每条轨迹的有效 Token 奖励求和，使用同组索引聚合奖励，再计算组内均值和标准差，最后把标量优势复制到响应序列长度并乘 Mask。若奖励管理器把一次成功写入某一个有效 Token，求和即可还原 R_i；从数学上不应理解为先把值 1 写入每个 Token、再把所有 Token 奖励求和成轨迹长度。这里区分的是*用于记录奖励的张量与用于优化的轨迹优势*。

2.2 Token-level GRPO 策略优化

旧策略与当前策略。设 $z_{i,q}$ 是第 i 条轨迹在有效位置 q 上由旧策略采样的动作 Token，对应模型条件输入是执行该 Chunk 时的观测 $s_{i,q}$，其中 q 同时隐含 Chunk 内的位置。新策略在同一位置、同一个已采样 *Token* 上的概率和旧策略概率的比值为

$$\rho_{i,q}(\theta) = \frac{\pi_\theta(z_{i,q} \mid s_{i,q})}{\pi_{\theta_{\text{old}}}(z_{i,q} \mid s_{i,q})} = \exp\big(\log p_{i,q}^{\text{new}} - \log p_{i,q}^{\text{old}}\big). \tag{2.4}$$

这里只重算给定采样 Token 的概率，不在训练 Loss 计算时重新抽取另一个 Token。旧策略 log probability 固定为采样时的数值；新策略概率保留梯度。比值使有限次数采样得到的同一批轨迹能够用于受约束的策略更新。

GRPO 裁剪目标。令 $\varepsilon_{\text{low}} = 0.2$、$\varepsilon_{\text{high}} = 0.28$，定义 Token 目标项

$$F_{i,q}(\theta) = \min\Big(\rho_{i,q}(\theta)\hat{A}_i,\ \text{clip}\big(\rho_{i,q}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\big)\,\hat{A}_i\Big). \tag{2.5}$$

总目标写为

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{L_i}\sum_{q=1}^{L_i} F_{i,q}(\theta)\right], \qquad \max_\theta \mathcal{J}(\theta), \tag{2.6}$$

并附有动态采样约束 $0 < \sum_i R_i < G$。L_i 是轨迹中参与 Loss 的有效动作 Token 数目，$1/L_i$ 是论文公式中的逐轨迹长度归一化。论文在此目标中去掉了固定 Reference Policy 的 KL 惩罚；但 Ratio Clipping 仍保留，所以不能理解为完全没有策略变化约束。

正优势与负优势为什么裁剪不同。对于 $\hat{A}_i > 0$，当 $\rho_{i,q} > 1.28$ 时，min 选择上界对应的常数平台，进一步提高该 Token 概率不会继续增加目标。对于 $\hat{A}_i < 0$，当 $\rho_{i,q} < 0.8$ 时，目标同样进入平台，防止失败 Token 的概率被一次更新过度压低。负优势的位置即使出现 $\rho > 1.28$，仍可能产生惩罚梯度，因为此时不该把“概率被增加”视作可接受的更新。

把一个 Action Chunk 代入公式。LIBERO 中若动作块大小为 $H = 8, d = 7$，展平后共有 56 个 Token。若第 i 条轨迹一共包含三个完整 Chunk，则 $L_i = 3 \times 56 = 168$。令 $q = (b, h, j)$ 表示第 b 个 Chunk、第 h 个原子动作、第 j 个分量。对整个轨迹的贡献是

$$\mathcal{J}_i(\theta) = \frac{1}{168} \sum_{b=0}^{2} \sum_{h=0}^{7} \sum_{j=1}^{7} F_{i,(b,h,j)}(\theta). \tag{2.7}$$

每个 Token 有自己的 $\rho_{i,(b,h,j)}$，但是整条轨迹共享 $\hat{A}_i$。因此不是先将 56 个 Ratio 相乘、再对乘积做一次 Clipping；它是 *Token* 逐项裁剪、轨迹聚合。

取成功轨迹 i 的优势 $\hat{A}_i = 0.9354$。其 Chunk 中一个 Token 的旧、新概率分别为 0.20、0.24，则 $\rho = 1.2$，有

$$F_{i,q} = \min(1.2 \times 0.9354, 1.2 \times 0.9354) = 1.1225. \tag{2.8}$$

另一个 Token 的旧、新概率分别为 0.05、0.08，则 $\rho = 1.6$，裁剪后

$$F_{i,q} = \min(1.6 \times 0.9354, 1.28 \times 0.9354) = 1.1973. \tag{2.9}$$

后一个 Token 处于正优势上侧裁剪平台，其概率进一步增大对这一项的局部梯度为零。失败轨迹使用负优势；在没有触发下侧裁剪时，优化会降低被采样 Token 的概率。这不是按物理动作正确与否逐步标注，而是利用整个任务的结果调整轨迹中动作的出现倾向。

梯度如何进入 VLA。在未触发裁剪的平台区域，单个 Token 的负目标为 $\ell_{i,q} = -\hat{A}_i \rho_{i,q}$，所以

$$\frac{\partial \ell_{i,q}}{\partial \log \pi_\theta(z_{i,q} \mid s_{i,q})} = -\hat{A}_i \rho_{i,q}. \tag{2.10}$$

若这一 Token 的预测 logits 为 u_k，在温度为 T 的 Softmax 下，采样 Token 为 z 时

$$\frac{\partial \log \pi_\theta(z \mid s)}{\partial u_k} = \frac{1}{T}\big(\mathbb{I}[k=z] - \pi_\theta(k \mid s)\big). \tag{2.11}$$

链式法则给出

$$\frac{\partial \ell_{i,q}}{\partial u_k} = -\frac{\hat{A}_i \rho_{i,q}}{T}\big(\mathbb{I}[k=z] - \pi_\theta(k \mid s)\big). \tag{2.12}$$

计入整体目标时，还需要乘 $1/(GL_i)$ 因子。正优势倾向提高被采样 Token 的概率，负优势倾向压低它。梯度从 Token logits 传回 LLaMA2 输出头及其他可训练的 Backbone 参数；论文使用全参数 RL 微调。环境产生状态和奖励，却不位于该损失函数的反向传播图中。

损失函数实现。公开实现的 `compute_policy_loss` 先计算新旧 log probability 的差，再取指数得到 Ratio；根据正负优势与不对称范围构造两种负目标，逐 Token 取最大，最后使用有效 Token Mask 求均值。将对应计算压缩为如下代码表达：

```
# logp_new, logp_old, advantage, valid: [B, L]
ratio = torch.exp(logp_new - logp_old)
raw = -advantage * ratio
clipped = -advantage * ratio.clamp(0.80, 1.28)
per_token_loss = torch.maximum(raw, clipped)
loss = (per_token_loss * valid).sum() / valid.sum()
```

其中 `valid` 为 0/1 有效动作 Mask。这与正文所述的符号方向一致：论文最大化 $\mathcal{J}$，代码最小化其负值。官方 `dp_rob.py` 进一步把长轨迹按小批次切片、计算各段损失、根据有效 Token 数加权并反向传播，优化器再更新 Actor 参数。应区分论文公式与实际批次聚合：公式对每条轨迹先取 $1/L_i$ 平均再组平均；核心代码使用 `masked_mean`，其权重与有效 Token 总数相关。不同长度轨迹混合时，两种归一化不严格等价。这个差别不会改变 Token-level Ratio 与 Clipping 的定义，但会影响轨迹之间的相对权重。

2.3 探索增强与完整训练闭环

Higher Rollout Temperature。训练期采样温度从 1.0 提高到 1.6。由式 (1.5) 可知，升温会降低大 logit 候选的相对优势，使其他 Token 更容易被选中。环境因此更可

能出现与当前主要动作模式不同的执行轨迹。探索更丰富未必意味着单次 Rollout 的成功率更高，它增加的是找到有价值成功行为的可能性。

Dynamic Sampling。一组完全成功或完全失败的轨迹不能通过组内标准化得到非零优势。论文因此只接纳同时包含两种结果的组，约束为

$$0 < \left|\{\tau_i : \text{is_successful}(\tau_i)\}\right| < G. \tag{2.13}$$

动态采样提升了进入更新阶段的组内比较信息量，但不会直接修复零成功率初始策略：如果某个任务根本无法采到成功轨迹，增加尝试次数也可能仍然没有有效组。该过程还改变了训练任务的样本分布，应区别于在无筛选条件下直接随机采样。

Clip Higher。原来的比值裁剪上限从 1.2 调整为 1.28，下限仍是 0.8。对于优势为正、旧策略概率较低的 Token，这允许更大的概率增长幅度；同时也可能让更新更激进。该改变只作用于策略优化阶段，不能代替温度采样，也不能创造原本不存在的成功轨迹。

一轮训练的数据流。已完成 SFT 的动作 Token 策略 π_{θ_k} 在任务初态上随机采样 G 条 Rollout，执行直到成功、失败或达到环境时长限制。环境给每条轨迹标注最终奖励；筛选存在成功和失败差异的组；计算 $\hat{A}_i$；对已采样的动作 Token 重新计算概率；以式 (2.6) 定义的裁剪目标更新 Actor。之后新策略用于下一轮环境交互：

$$\theta_k \xrightarrow{\text{rollout}} \{\tau_i\}_{i=1}^{G} \xrightarrow{\text{reward}} \{R_i\} \xrightarrow{\text{GRPO}} \theta_{k+1}. \tag{2.14}$$

从损失计算角度，涉及三个严格不同的时间尺度：动作块内部的原子控制步 h、整条轨迹内的 Chunk 次数，以及参数更新轮次 k。Rollout 是环境交互阶段；Policy Loss 只对收集好的 Token 概率求导；更新后新的权重再用于新的 Rollout。这是一种在线、闭环、结果驱动的 VLA 强化学习训练方式。

对于任务完成后不再存在的控制时间步，Mask 必须为零；评估阶段仅运行策略前向、动作解码与闭环控制，不使用奖励或 GRPO 更新。结果奖励将同一条失败轨迹中的全部有效 Token 赋予负优势，即使早期动作本身合理，也无法得到细粒度的正向反馈。这是轨迹级 Credit Assignment 的主要限制；但它不需要独立 Critic，能够直接对动作 Token 概率执行 GRPO。